COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Oct 6, 2026	**DATE SUBMITTED:**	Oct 7, 2026
NAME:	Isaiah Benedict R. Garbosa Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11 **Basic Algorithm Analysis**

1. PROCEDURE OUTPUT

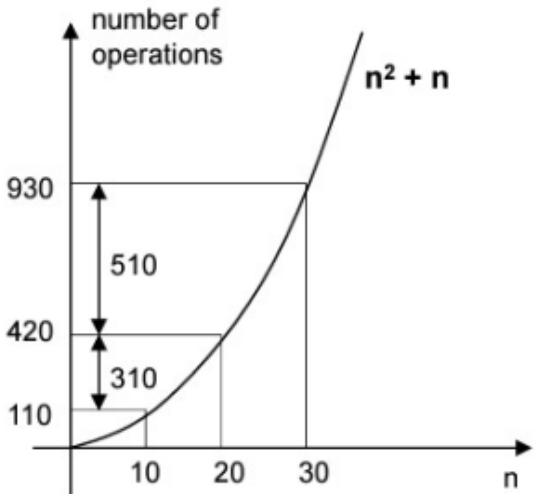
ILO A: Measure the runtime of algorithms using theoretical analysis

Worst Case

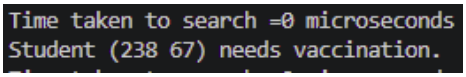
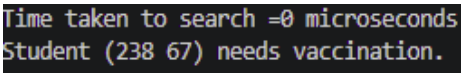
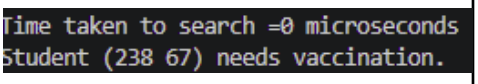
Assess if two arrays have any common values.

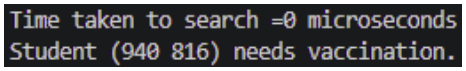
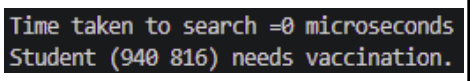
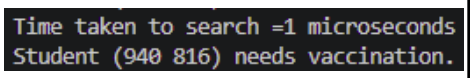
```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

The total number of comparisons in the worst case is: n^2 + n

Analysis
 | n (size of array)	number of operations | |---|---| | 10 | 110 | | 20 | 420 | | 30 | 930 | | | |
As the size of the array increases, the number of operations will increase quadratically. This is due to 2 for loops used to compare each element that are adjacent. The '+ n' is because of the printing of a boolean value after each comparison, and each comparison is equal to that of n.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Garbosa's Work			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1,000	0 microseconds		The execution speed for the input size of 1,000 is 0 microseconds.
10,000	0 microseconds		The execution speed for the input size of 10,000 is 0 microseconds.
100,000	0 microseconds		The execution speed for the input size of 100,000 is 0 microseconds.
Analysis			
The code implements a binary search to see if a student is present on a list, with each student being represented as an object of a student class. The needs_vaccination function searches through the list using binary search. The search_test function checks if a student is present on the list, if not, it will return a true. If a student is found on the list, it will return that the student needs vaccination and the time it takes to search for that student. For the final result, each input size return about the same execution speed at 0 microseconds.			

Vizcayno's Work			
Input Size (N)	**Execution Speed**	**Screenshot**	**Observation(s)**
1,000	0 microseconds		The time taken to search 1,000 data takes 0 microseconds.
10,000	0 microseconds		The time taken to search 10,000 data takes 0 microseconds.
100,000	1 microseconds		The time taken to search 100,000 data takes 1 microsecond.
Analysis			
The time it takes to search size n, increases as the size n increases. Compared to the first 2 input sizes, input size #3 took 1 microsecond more to search. This is because of the increase in size n. On the other hand, comparing the outcomes of both work, there is a difference in time taken to search in each given input size. This might be because of varying factors affecting the execution speed of both computers. This means that even if the code is efficient in theory, the speed also factors in the hardware used to process and execute information.			

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

Problem 1	Consider a program that returns true if the elements in an array are unique.

	`Algorithm Unique(A)`
	`  for i = 0 to n-1 do`
	`  for j = i+1 to n-1 do`
	`  if A[i] equals to A[j] then`
	`  return false`
	`  return true`

Theoretical Analysis	$T(n) = n^2 + n$

Experimental Analysis	`#include <iostream>`
	`#include <chrono>`
	
	`using namespace std;`
	
	`bool Unique(const int A[], int n, long long& op_count) {`
	`  for (int i = 0; i < n - 1; i++) {`
	`  for (int j = i + 1; j < n; j++) {`
	`  op_count++;`
	
	`  if (A[i] == A[j]) {`
	`  return false;`
	`  }`
	`  }`
	`  }`
	
	`  return true;`
	`}`

```cpp
int main() {
    int sizes[] = {1000, 2000, 4000, 8000, 16000};

    for (int n : sizes) {
        int* A = new int[n];

        for (int i = 0; i < n; i++) {
            A[i] = i;
        }

        long long op_count = 0;

        auto start = chrono::high_resolution_clock::now();

        Unique(A, n, op_count);

        auto end = chrono::high_resolution_clock::now();

        auto time = chrono::duration_cast<chrono::nanoseconds>
                    (end - start).count();

        cout << "N = " << n
             << "\t\tOperations = " << op_count
             << "\t\tTime = " << time << " ms" << endl;

        delete[] A;
    }

    return 0;
}
```

```
N = 1000        Operations = 499500        Time = 1279500 ms
N = 2000        Operations = 1999000       Time = 7269800 ms
N = 4000        Operations = 7998000       Time = 20052100 ms
N = 8000        Operations = 31996000      Time = 79662200 ms
N = 16000       Operations = 127992000     Time = 312611200 ms
```

Analysis and Comparison

The result of the experimental analysis support the theoretical analysis since the algorithm has a quadratic running time, $O(n^2)$. It is noticeable that as the input size increases, the number of operations and execution time also increases significantly. When the input sizes double, the number of operations increases four times since the code utilizes two for loops. It shows the quadratic growth of the algorithm. Overall, the experimental analysis was able to be consistent with the theoretical analysis, which confirms that the algorithm becomes slower as the input size increases.

Problem 2	Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis	**rpower algorithm** : T (n) = n **brpower algorithm** : T (n) = log n

Experimental Analysis

```cpp
#include <iostream>
#include <chrono>
#include <cmath>

using namespace std;

// Algorithm rpwr
long long rpwr(long long x, int n, long long& op_count) {
    op_count++;

    if (n == 0)
        return 1;

    return x * rpwr(x, n - 1, op_count);
}

// Algorithm brpower
long long brpower(long long x, int n, long long& op_count) {
    op_count++;

    if (n == 0)
        return 1;

    if (n % 2 == 1) {
        long long y = brpower(x, (n - 1) / 2, op_count);
        return x * y * y;
    }
    else {
        long long y = brpower(x, n / 2, op_count);
        return y * y;
    }
}
```

```
int main() {

    int sizes[] = {1000, 2000, 4000, 8000, 16000};

    cout << "RPWR:" << endl;

    for (int n : sizes) {

        long long op_count = 0;

        auto start = chrono::high_resolution_clock::now();

        rpwr(2, n, op_count);

        auto end = chrono::high_resolution_clock::now();

        auto time = chrono::duration_cast<chrono::nanoseconds>
                    (end - start).count();

        cout << "N = " << n
             << "\tOperations = " << op_count
             << "\tTime = " << time << " ms" << endl;
    }

    cout << endl << "BRPOWER:" << endl;
```

```
    for (int n : sizes) {

        long long op_count = 0;

        auto start = chrono::high_resolution_clock::now();

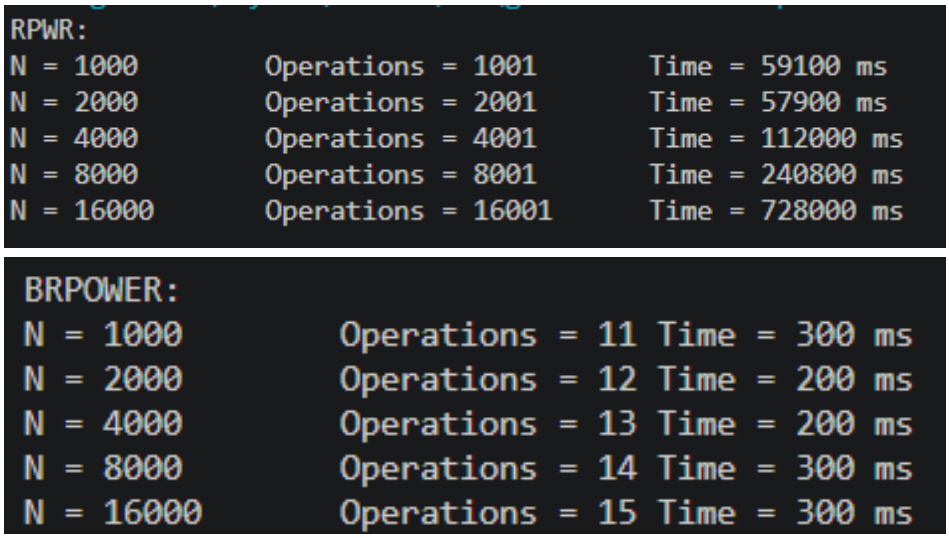
        brpower(2, n, op_count);

        auto end = chrono::high_resolution_clock::now();

        auto time = chrono::duration_cast<chrono::nanoseconds>
                    (end - start).count();

        cout << "N = " << n
             << "\tOperations = " << op_count
             << "\tTime = " << time << " ms" << endl;
    }

    return 0;
}
```

	
Analysis and Comparison	The results of the experimental analysis also supports the theoretical analysis of both of the algorithms. For the rpwr algorithm, it has an O(n) time complexity, and its operation increases directly with the value of n, which results in longer execution times as n increases. On the other hand, brpower has O(log n) time complexity, which requires only a small increase in operations even if n increases. Its execution time is also fairly consistent. Therefore, it shows that brpower is significantly more efficient than rpwr, especially when working with large values of n. This is consistent with the theoretical analysis as well.

Conclusion - Garbosa
In this activity, I was able to learn and practice basic algorithm analysis. I was able to learn about theoretical analysis and experimental analysis. I learned about how different functions and methods can vary in their runtime and number of operations. Functions with quadratic algorithms O(n^2), can become less effective once the input becomes larger. Logarithmic functions are more effective for larger size inputs since it requires only a small increase in operations. Furthermore, an extra thing that I have learned is that experimental analysis depends too much on hardware. Time complexities can vary based on the hardware. Some computers can run some algorithms faster and some run them slower. Overall, I believe that I was able to learn the basics of algorithm analysis and I am able to determine the time complexity of simple algorithms. However, there is still a lot more to improve since I am not too familiar with the code that was used in this activity yet and I think that I would struggle on more complex algorithms.

Conclusion - Vizcayno
The activity discusses that when finding the execution time of a program, there are several factors that affect its speed. These factors can come from the hardware and/or software environment where the program is run. Because of this, the same program can take different times on different computers, so we look at how the number of operations grows as the number of elements increases. In measuring the execution speed, we can use

theoretical and experimental analysis. The theoretical analysis is a way for us to estimate the number of operations in terms of the input size n, without running the program. The experimental analysis, on the other hand, is when we run the program and measure the time it takes. As long as we count the same kind of operations for each algorithm, comparing them is fair.

So, this activity allowed me to estimate, measure, and understand how running time analysis is helpful in determining the efficiency of a program. The theoretical analysis showed me how the number of operations grows as the input size increases, while the experimental analysis showed me the actual time the program took on the device I used. For example, in ILO B, I ran the same binary search program and compared my results with my groupmate. The first 2 results were the same, but for the 3rd result, mine took 1 microsecond longer. This shows that the measured time can differ a little even when the program is the same, but the growth of the operations stays the same.

The activity also showed that as the size of the data increases, the number of operations also increases, but how fast it grows depends on the algorithm. I was able to complete this activity faster because I had someone to compare thoughts and observations with. While I could understand the lesson, it takes me a while to comprehend new libraries since different libraries are used and implemented in different ways. I think I should take the initiative to explore various libraries so I can familiarize myself with their functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to check for grammar mistakes or fix the flow of the sentence or paragraph.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.